

《魔兽世界》编程大作业辅助工具——项目设计报告

目录

一、程序功能介绍	1
1.1 程序整体定位	1
1.2 核心功能说明	1
二、项目各模块与类设计细节	2
2.1 引擎层 (engine/)	2
2.2 界面层 (ui/)	5
2.3 架构优化价值分析	7
三、小组成员分工情况	7
3.1 黄宇曦	7
3.2 甘城屹	8
3.3 文卓远	8
四、项目总结与反思	8
4.1 项目实施历程	9
4.2 主要技术难点与解决方案	9
4.3 团队协作与工程实践收获	10

一、程序功能介绍

本项目开发了一个基于 PySide6 的桌面端编程学习辅助工具，围绕《程序设计实习》课程中经典的“魔兽世界终极版”(Warriors4) C++ 大作业，为学生提供从类结构设计、模拟仿真验证到日志诊断调错的完整学习闭环。

1.1 程序整体定位

程序以图形化交互界面 (GUI) 组织五大核心功能模块，覆盖 C++ 编程大作业的三个子任务 (Task1 ~ Task3)。程序的入口文件 `main.py` 负责启动 PySide6 应用主窗口，命令行入口文件 `run_warriors4.py` 则提供 OJ 格式输入文件的批量验证能力，两者共享 `engine/` 中的纯业务逻辑代码。

1.2 核心功能说明

(1) 可视化类编辑器

提供 C++ 类结构的可视化设计能力，支持新建/删除类、指定基类、添加/拖拽排序成员变量、设置虚函数等操作。右侧面板实时预览生成的 C++ 类定义代码，完成后可通过菜单导出。

(2) 对象内存布局视图

基于类定义自动计算对象内存布局（含虚表指针、成员偏移量、对齐填充），以彩色条形图形式可视化展示。支持悬停查看字段的偏移、大小和来源类，帮助学生理解 C++ 对象模型中的内存对齐与继承布局。

(3) Task1——武士类层级校验

采用积木式编程（Block Programming）范式，对学生设计的 `Warrior` 及其子类（`Dragon` / `Ninja` / `Iceman`）进行自动化校验。校验内容包括：必要成员变量是否存在（`hp`、`attack`）、直接继承关系是否正确、构造函数初始化列表是否调用基类构造、虚函数声明是否合理等。校验结果以成功/警告/错误三级提示呈现。

(4) Task2——整局战斗模拟引擎

依据题目规则（详见 `t.txt`），实现完整的“魔兽世界”回合制战斗模拟。核心机制包括：

- 参数配置：支持设置司令部生命元 `M`、城市数 `N`、箭攻击力 `R`、忠诚度衰减 `K`、时间上限 `T`，以及五种武士的生命值表和攻击力表。
- 事件阶段：严格按照题面定义的每小时 10 个阶段（00' 造兵、05' 狮子逃跑、10' 行军、20' 城市产出、30' 收集生命元、35' 放箭、38' 炸弹自爆、40' 战斗、50' 司令部报告、55' 武器报告）推进模拟。
- 教学/标准双模式：教学模式允许启用/禁用特定阶段并调整阶段分钟值，便于分步理解；标准模式锁定题面配置。
- 战斗系统：完整实现攻击-反击流程、武器（剑/箭/炸弹）使用与钝化规则、士气与忠诚度系统、城市旗帜机制、生命元奖励与回收逻辑。
- C++ 代码导出：支持两种导出模式——OJ 单文件题解（`task2_solution.cpp`，可直接提交）和模块化学习工程骨架（按 `Game/Headquarter/City/Warrior/Weapon` 拆分的 `.h/.cpp` 文件），两种模式均可用 `g++ -std=c++20` 编译运行。
- 内置时间轴播放器：在 Task2 面板中集成时间轴控件，支持播放/暂停/步进/跳转，便于复盘事件顺序和局部状态。

(5) Task3——日志对拍与 AI 辅助调试

- 接收 Task2 导出的模拟事件流，生成标准格式日志文本。
- 支持按小时、阶段、关键词筛选局部事件。
- 将学生程序输出与标准日志逐行对比，以 HTML 格式高亮差异。
- 集成多模型 AI 调试助手（支持 DeepSeek Chat、OpenAI GPT-4o mini、GPT-4.1 mini 及自定义 OpenAI 兼容接口），将差异上下文发送给大语言模型，返回中文调试建议。API Key 仅用于当次请求，不持久化存储。

(6) 辅助功能

- 简化战斗实验室: `battle_engine.py` 提供独立的回合制战斗模拟, 可用于课堂演示攻击-反击流程。
- 环境自检工具: `check_environment.py` 检测 Python 解释器类型 (CPython/PyPy)、PySide6 安装状态、g++ 可用性, 并给出下一步建议。
- 一键回归验证: `run_regressions.py` 集成多项自动化检查, 包括 Python 语法编译、GUI 模块导入、引擎层无 GUI 依赖验证、默认类设计覆盖率、C++ 骨架编译运行、Warriors4 样例回归等。

二、项目各模块与类设计细节

本项目的架构实现了严格的前后端分离: `engine/` 为纯业务逻辑层, 不依赖任何 GUI 框架; `ui/` 为基于 PySide6 的界面交互层。两层之间通过明确的信号-槽机制和数据对象传递进行解耦通信。

2.1 引擎层 (`engine/`)

引擎层包含 15 个 Python 模块, 各司其职:

2.1.1 核心数据模型 `models.py` —— 通用数据类定义

数据类	用途
<code>MemberDef</code>	成员变量定义 (名称、类型、注释)
<code>ClassDef</code>	类定义 (名称、基类、成员列表、是否虚类)
<code>MemoryBlock</code>	内存块 (偏移、大小、类型、来源类), 含 <code>source_class</code> 字段追踪继承来源
<code>ObjectInstance</code>	对象实例 (所属类、内存块列表、总大小)

`warcraft_models.py` —— 魔兽世界仿真数据结构

核心数据类	职责
<code>StageDefinition</code>	标准事件阶段枚举 (00/05/10/20/30/35/38/40/50/55 分钟)
<code>EventSlotConfig</code>	可配置的阶段槽位 (启用/禁用、分钟值覆盖)
<code>EventScheduleProfile</code>	阶段调度配置文件 (含 <code>strict_mode</code> 标志)

核心数据类	职责
WarcraftConfig	全局配置 (M/N/R/K/T 参数、生命值表、攻击力表)
WeaponSet	武器集合 (剑/箭/炸弹), 含钝化、消耗、缴获逻辑
WarriorUnit	武士实体 (阵营、种类、编号、生命值、攻击力、位置、武器、士气、忠诚度、步数、移动履历)
HeadquarterState	司令部状态 (生命元、制造序列、已生产计数、待奖励列表)
CityState	城市状态 (生命元、旗帜、连胜记录)
EventRecord	事件记录 (时间、阶段键、位置、文本描述)
StageExecution	阶段执行结果封装
SimulationBundle	完整模拟打包 (配置、调度、事件列表、最终世界状态)

2.1.2 类管理与代码生成子系统 `class_manager.py` —— `ClassManager` 单例类

这是类管理子系统的中枢, 职责包括: - 类定义的 CRUD 操作 (`add_class` / `update_class` / `remove_class` / `get_class`) - 继承关系管理: `is_subclass_of()` 判定继承链、`get_all_members()` 合并基类成员、`class_has_virtual()` 虚函数检测、循环继承检测 - 内存布局计算: `compute_memory_layout()` 计算虚表指针、成员偏移量、对齐填充, 返回含 `source_class` 来源标注的 `MemoryBlock` 列表 - C++ 代码生成: `generate_cpp_code()` 生成单个类的 C++ 声明、`export_all_cpp()` 导出单文件头、`export_cpp_skeleton()` 导出多文件工程骨架 - `seed_demo_classes()`: 预置题目所需全部实体类模板 (Headquarter、City、Weapon 及其子类 Sword/Bomb/Arrow、Warrior 及其子类 Dragon/Ninja/Iceman/Lion/Wolf)

`task2_cpp_exporter.py`

负责将 Python 参考模拟器的算法逻辑翻译为可编译的 C++ 代码。内部包含完整的 C++ 引擎实现字符串 (约 1500 行), 生成 `task2_solution.cpp` (OJ 单文件提交版) 及模块化骨架工程 (`Game.h/cpp`、`Headquarter.h/cpp`、`City.h/cpp`、`Warrior.h/cpp`、`Weapon.h/cpp`、`main.cpp`、`README.txt`、`MODULE_DESIGN.md`)。

2.1.3 Task1 校验子系统 `task1_validator.py`

- `StandardClassRule`: 定义标准类的直接基类和必要成员字段要求
- `ValidationResult`: 校验结果 (等级: ok / success / warning / error)
- `validate_warrior_hierarchy()`: 校验 Warrior 及子类的继承结构和成员完整性
- `validate_warcraft_entities()`: 检查是否覆盖题目全部实体类要求

2.1.4 Task2 模拟引擎子系统 以 `warcraft_engine.py` 中的 `WarcraftEngine` 为核心，通过职责委托协调以下模块：

子模块	核心类/函数	职责
<code>warcraft_factory.py</code>	<code>create_next_warrior()</code> , <code>build_initial_weapons()</code>	武士按阵营顺序创建、初始武器分配
<code>warcraft_queries.py</code>	<code>alive_warriors()</code> , <code>alive_warriors_at()</code> , <code>first_alive_enemy_at()</code> 等	战场状态查询
<code>warcraft_battle_rules.py</code>	<code>resolve_attacker()</code> , <code>predict_battle_deaths()</code> , <code>update_city_flag()</code>	先手判定、死亡预测、旗帜规则
<code>warcraft_reporting.py</code>	<code>format_march()</code> , <code>format_headquarter_reached()</code>	输出文本格式化
<code>warcraft_models.py</code>	(见上文数据类表)	数据模型与配置

`WarcraftEngine` 提供的核心接口：

- `reset()` / `initialize_case()`：初始化模拟状态
- `next_stage()`：推进单个事件阶段
- `run_next_hour()`：运行完整一小时的 10 个阶段
- `run_until_limit()`：运行至时间上限或游戏结束
- `export_bundle()`：导出 `SimulationBundle` (供 Task3 消费)

十个阶段方法 (`_run_spawn_stage`、`_run_lion_escape_stage`、`_run_march_stage` 等) 各自封装一个时间片的游戏逻辑，便于独立测试和维护。

`warriors4_runner.py`

- `Warriors4Case`：OJ 格式的测试用例数据结构
- `parse_warriors4_input()`：从 OJ 输入文本解析用例列表
- `render_warriors4_output()`：调用 `WarcraftEngine` 运行用例并生成 `Case n`：格式输出
- `solve_warriors4_file()`：端到端文件处理入口

2.1.5 Task3 日志子系统与 AI 助手 `task3_log_helper.py`: 日志构建 (`build_log_text()`)、事件筛选 (`filter_events()`)、逐行比较 (`compare_logs()`)，基于 `difflib.HtmlDiff` 生成 HTML 差异报告)。

`ai_log_assistant.py`: 多模型 AI 调试接口。`build_model_config()` 验证并构建 API 配置, `explain_log_mismatch()` 将日志差异上下文发送给大语言模型并返回中文调试建议。当日志完全匹配时短路返回，避免不必要的 API 调用。

2.1.6 辅助模块 `battle_engine.py`: `BattleEngine` 类提供独立的简化版回合制战斗模拟，含攻击-反击流程、武器退化、战斗事件记录，可用于课堂演示或战斗实验室功能。

2.2 界面层 (ui/)

2.2.1 主窗口与布局 `mainwindow.py` —— `MainWindow(QMainWindow)`

顶层窗口，以 `QTabWidget` 组织 5 个标签页（类编辑器、内存视图、Task1、Task2、Task3）。通过信号-槽连接 Task2 的 `eventsExported` 信号至 Task3 的 `load_simulation_bundle` 槽，实现跨标签页的事件流传递。菜单栏提供 C++ 导出入口、版本信息和题面介绍对话框。

2.2.2 功能组件

组件文件	核心类	功能
<code>class_editor_widget.py</code>	<code>ClassEditorWidget</code> , <code>MemberTableWidget</code> , <code>TypePaletteList</code>	三栏布局的类编辑器，支持拖拽排序
<code>memory_view_widget.py</code>	<code>MemoryViewWidget</code> , <code>MemoryCanvas</code>	内存布局彩色条形图，支持悬停提示和点击选中
<code>task1_widget.py</code>	<code>Task1Widget</code>	类层级校验界面，集成积木编程编辑器
<code>task2_widget.py</code>	<code>Task2Widget</code>	模拟器控制面板，内嵌时间轴和世界状态显示
<code>task3_widget.py</code>	<code>Task3Widget</code>	日志对拍与 AI 调错界面，内嵌时间轴
<code>timeline_panel.py</code>	<code>TimelinePanel</code>	可复用时间轴控件（播放/暂停/步进/滑块）

组件文件	核心类	功能
<code>timeline_controller.py</code>	<code>TimelineController(QObject)</code>	时间轴播放控制器，1 秒定时器驱动自动步进
<code>block_workspace.py</code>	<code>BlockProgramEditor</code> , <code>BlockPaletteList</code> , <code>BlockWorkspaceList</code>	积木式编程框架，支持拖拽编排和脚本序列化

2.2.3 积木式编程框架 (Block Programming) `block_workspace.py` 实现了一套通用的可视化脚本编排框架，供 Task1、Task2、Task3 复用。核心设计：

- `BlockSpec / FieldSpec`：定义积木块的元数据（名称、描述、字段类型及参数）
- `BlockPaletteList`：可拖拽的积木候选区
- `BlockWorkspaceList`：可放置和重排的积木工作区
- `BlockItemWidget`：单个积木块的渲染组件（彩色标识条、徽章、标题、描述文字、字段控件）
- `BlockProgramEditor`：组合候选区与工作区，提供脚本序列化/反序列化（`get_script()` / `set_script()`）

2.2.4 UI 生成文件 `ui_form.py` 和 `ui_dialog.py` 由 Qt Designer 自动生成，定义了窗口框架结构；`resources_rc.py` 包含嵌入的 SVG 图标资源。

2.3 架构优化价值分析

与初版相比，第二版（当前版本）的模块化重构带来了以下显著改进：

(1) 关注点分离 (Separation of Concerns)

引擎层与界面层的物理隔离（`run_regressions.py` 中通过 `check_engine_layer_is_gui_free` 断言 `engine/` 不含任何 `PySide6` 导入），使得：- 引擎逻辑可以脱离 GUI 独立运行和测试（例如通过 `python run_warriors4.py` 命令行验证）- 两端开发可并行进行，降低耦合风险

(2) 引擎层的细粒度拆分

Task2 模拟引擎从单一文件转变为 7 个模块（`warcraft_models`、`warcraft_factory`、`warcraft_queries`、`warcraft_battle_rules`、`warcraft_reporting`、`warcraft_engine`、`warriors4_runner`），每个模块职责单一、接口清晰。这确保了在修改战斗规则（如先手判定或旗帜逻辑）时，不会意外影响数据模型或文本格式化。

(3) UI 组件的可复用设计

时间轴面板（`TimelinePanel` + `TimelineController`）和积木编程框架（`block_workspace.py`）都设计为可复

用组件，避免了 Task1/Task2/Task3 中重复实现相似功能。当需要在 Task3 中嵌入时间轴时，仅需实例化已有组件并连接数据源，无需重写。

(4) 代码生成的分离

`task2_cpp_exporter.py` 将 C++ 代码生成逻辑独立为专门模块，内部维护完整的嵌入式 C++ 引擎实现字符串。Python 参考引擎的架构变更不影响 C++ 导出逻辑，反之亦然。

(5) 回归验证体系的建立

`run_regressions.py` 提供的自动化验证体系是第二版新增的质量保障措施，覆盖编译检查、模块边界验证、C++ 交叉编译验证和样例回归，确保每次修改不会破坏已有功能。

三、小组成员分工情况

3.1 黄宇曦

具体分工：类继承校验、战斗模拟引擎与日志对齐系统

- (1) 实现 Task1 校验模块 (`task1_validator.py` / `task1_widget.py`)，依托 `ClassManager` 对 `Dragon` / `Ninja` / `Iceman` 进行继承链校验（基类 `Warrior`）、必要字段（`hp` / `attack`）检查及构造函数初始化列表的语义比对，并提供明确的错误定位与成功反馈。
- (2) 开发 Task2 回合制战斗引擎 (`battle_engine.py` / `task2_battle.py` / `task2_widget.py`)，支持从 `ClassManager` 动态下拉选取武士实例、逐步或自动执行战斗推演，实时处理攻防计算、反击机制与武器耐久损耗，输出结构化战斗日志至 `QTextEdit`，并提供“导出事件到时间轴”按钮，调用成员 A 的 `load_events` 接口生成可视化时间轴。
- (3) 构建 Task3 日志对齐助手 (`task3_log_helper.py` / `task3_widget.py`)，内置严格遵循 `%03d:%02d` 时间戳规范的标准日志生成器，支持用户粘贴程序输出文本并通过字符级 / 行级高亮差异比对，精准辅助调试。

3.2 甘城屹

具体分工：UI 框架与时间轴模块

- (1) 构建主窗口框架 (`ui/mainwindow.py`)，使用 `QTabWidget` 组织五大功能标签页（类编辑器、内存视图、Task1、Task2、Task3），实现菜单栏（文件-导出 C++ 代码、帮助-关于对话框）。
- (2) 开发可复用时间轴组件 (`ui/timeline_panel.py`)，集成 `QSlider` 进度控制、播放 / 暂停 / 步进 / 快进按钮、事件详情显示区 (`QTextEdit`)，通过 `QTimer` 实现自动播放。
- (3) 实现时间轴控制器 (`ui/timeline_controller.py`)，提供 `set_events(events)` 接口接收外部事件流，支

持跳转至指定时间点、单步回溯和变速播放。

- (4) 在主窗口中通过信号-槽机制连接各模块，实现 Task2 模拟器导出事件流至时间轴的联动（`eventsExported` 信号连接至 `load_events` 槽）。
- (5) 预置魔兽世界示例事件数据，用于时间轴功能演示。

3.3 文卓远

具体分工：类结构编辑器与内存切片视图

- (1) 设计并实现 `ClassManager` 单例类（`engine/class_manager.py`），提供类定义的 CRUD 操作、继承链判定（`is_subclass_of`）、虚函数检测、循环继承检测、C++ 代码生成（`generate_cpp_code` / `export_all_cpp` / `export_cpp_skeleton`）以及内存布局计算（`compute_memory_layout`，含虚表指针、对齐填充、来源类追踪）。
- (2) 开发可视化类编辑器（`ui/class_editor_widget.py`），实现三栏布局（类列表 / 编辑表单 / C++ 代码实时预览），支持拖拽排序成员变量和积木式类型拼接。
- (3) 实现内存布局视图（`ui/memory_view_widget.py`），基于 `QPainter` 绘制彩色内存条形图，支持悬停查看偏移量、大小和来源类，直观展示 C++ 对象模型。

四、项目总结与反思

4.1 项目实施历程

本项目经历了“从零到一、再迭代优化”的完整开发过程。

第一阶段——需求理解与原型搭建

项目伊始，我们首先深入研读了《魔兽世界终极版》的题面文本（`t.txt`），梳理出 5 种武士类型、3 种武器、10 个事件阶段（00' ~ 55'）以及复杂的战斗规则（攻击-反击流程、武器钝化、士气/忠诚度系统、城市旗帜机制等）。在此基础上，我们搭建了第一版原型，实现了基本的程序逻辑——武士创建、行军、战斗和事件输出。初版的核心目标是“跑通流程”，代码结构较为扁平，业务逻辑与界面代码交织在一起。

第二阶段——架构重构与功能完善

在原型验证可行的基础上，我们进行了全面的架构重构。核心思路是：将程序分为 `engine/`（纯业务逻辑）和 `ui/`（图形界面）两大层，并在 `engine` 内部进一步按子任务和功能域拆分模块。同时，我们新增了多项功能：

- 积木式编程：将类校验、模拟配置和日志分析抽象为可组合的积木块，提升交互灵活性
- 时间轴回放：从独立标签页改为内嵌于 Task2/Task3，提供更直观的事件流程复盘体验

- **AI 辅助调试**：集成多模型 API 调用，利用大语言模型帮助学生快速定位日志格式或逻辑差异
- **OJ 一键验证与回归测试**：提供命令行入口和自动化回归套件，确保代码质量

4.2 主要技术难点与解决方案

(1) 战斗系统的完整性与正确性

魔兽世界终极版的战斗机制极为复杂：先手判定依据城市旗帜和奇偶性、反击不触发武器钝化、弓箭可射杀相邻城市敌人、炸弹预判致死触发同归于尽、Dragon 的士气在战斗前后变化、Lion 的战死转移生命值、Wolf 的武器缴获等，规则交织且彼此影响。

解决方案：我们将战斗逻辑拆分为独立的 `warcraft_battle_rules.py` 模块，用 `resolve_attacker()`、`predict_battle_deaths()`、`update_city_flag()` 三个纯函数封装核心判定逻辑，便于单独编写测试用例验证。战斗编排（调用顺序、事件记录）则由 `WarcraftEngine._simulate_battle()` 统一管理。

(2) 内存布局计算的正确性

内存视图需要正确反映 C++ 对象模型：虚表指针位置、基类子对象布局、成员对齐填充等。这与题目要求直接相关——学生需要理解 `sizeof` 的结果以及成员偏移量。

解决方案：`ClassManager.compute_memory_layout()` 采用递归遍历继承链的方式合并成员，在模拟编译器行为时遵循“先基类后派生类、同访问控制符内按声明顺序、按最大对齐要求插入填充”的原则。每个 `MemoryBlock` 标注 `source_class`，使继承来源可视化。

(3) C++ 代码导出的正确性

OJ 单文件题解需要在单个 `.cpp` 文件中嵌入完整的模拟引擎，同时保持 `g++ -std=c++20` 可编译。Python 参考实现与 C++ 导出代码需要保持语义等价。

解决方案：`task2_cpp_exporter.py` 内部维护了完整的 C++ 引擎实现字符串（约 1500 行），结构上与 Python 引擎模块一一对应（`WarcraftConfig`、`EventScheduleProfile`、`WarriorUnit`、`HeadquarterState`、`CityState`、`WarcraftEngine` 等）。回归测试套件中的 `check_task2_cpp_skeleton_compiles_and_runs()` 确保每次修改后 C++ 导出仍然通过编译和执行验证。

(4) GUI 与引擎的解耦

初版中界面代码直接调用引擎函数，导致引擎层的修改频繁影响 GUI，测试也难以进行。

解决方案：通过信号-槽机制（Qt Signal-Slot）实现解耦。例如，Task2 完成模拟后发出 `eventsExported` 信号，Task3 通过 `load_simulation_bundle` 槽接收数据，两个组件之间无需直接引用。引擎层完全无 PySide6 依赖，可通过 `run_regressions.py` 独立验证。

4.3 团队协作与工程实践收获

通过本次项目，我们在以下方面获得了实质性的提升：

1. 大型程序架构设计能力：从初版的”代码堆砌”到第二版的”模块化分层架构”，我们亲身体验了良好架构对代码可维护性、可测试性和可扩展性的关键作用。特别是”引擎-界面分离”和”职责单一原则”的实践，使我们对软件工程中的关注点分离有了切身体会。
2. 复杂规则系统的建模方法：魔兽世界的规则集是一个典型的复杂领域模型。我们学会了如何将自然语言描述的规则系统性地转化为数据模型和算法，并在实现过程中保持对题面的忠实还原。
3. 测试驱动与自动化验证意识：`run_regressions.py`的建立确保了每次代码变更都有自动化保障。我们认识到，对于多模块、多人协作的项目，自动化回归测试不是”可选项”，而是”必需品”。
4. 工具链集成能力：项目涉及 Python (PySide6 GUI)、C++ (g++ 编译验证)、AI API 调用等多种技术栈的集成，锻炼了我们在异构技术之间搭建桥梁的能力。
5. 文档与代码一致性维护：`README.md`详细记录了每个功能的使用方法、项目结构和开发验证流程，确保了团队成员和后续使用者能够快速上手。

附注：本报告基于项目第二版 (`feature-task2-finer-modules` 分支) 代码编写。项目源码结构为 `engine/` (15 个模块, 纯业务逻辑)、`ui/` (12 个模块, PySide6 界面)、`warriors4_data/` (测试数据), 入口文件为 `main.py` (GUI) 和 `run_warriors4.py` (命令行), 自动化验证入口为 `run_regressions.py`。